

INFORME DE PROYECTO: APPFIRE - GESTIÓN DE ACTIVIDADES

1. Introducción

En la actualidad, la organización personal y académica es fundamental para el éxito de cualquier estudiante. Con el aumento de responsabilidades, surge la necesidad de contar con herramientas digitales que permitan centralizar tareas, exámenes y proyectos en un solo lugar. El presente informe detalla el desarrollo de AppFire, una aplicación móvil desarrollada para el ecosistema Android utilizando las tecnologías más modernas de Google: Jetpack Compose para la interfaz y Firebase para la persistencia de datos en la nube.

2. Problema resuelto

Muchos estudiantes suelen dispersar su información en notas físicas, calendarios o aplicaciones que no se sincronizan en tiempo real. Esto genera desorden, olvido de fechas importantes y falta de seguimiento sobre qué tareas han sido completadas. AppFire resuelve este problema ofreciendo una plataforma centralizada, accesible desde cualquier dispositivo Android, que permite gestionar el ciclo de vida de una actividad académica de forma inmediata.

3. Objetivo de la app

El objetivo principal de AppFire es proporcionar una herramienta sencilla, intuitiva y eficaz para el registro y seguimiento de actividades académicas. La aplicación busca que el usuario pueda categorizar sus pendientes (Exámenes, Tareas, Proyectos), visualizar su estado de avance y mantener la información respaldada de forma segura en la nube mediante una base de datos no relacional.

4. Descripción del flujo de la aplicación

El flujo de la aplicación ha sido diseñado para ser directo y sin fricciones:

1. Pantalla de Inicio (Lista): Al abrir la app, el usuario visualiza todas sus actividades guardadas. Los datos se cargan en tiempo real desde Firebase.
2. Pantalla de Registro (Formulario): Mediante un botón de acción, el usuario accede a un formulario donde ingresa el título, descripción, selecciona el tipo de actividad mediante botones de opción (RadioButton) y marca si ya ha sido completada.
3. Edición: Al pulsar sobre una actividad existente, el sistema redirige al mismo formulario, pero precarga los datos automáticamente para permitir su modificación.
4. Eliminación: El usuario puede eliminar tareas que ya no necesite desde la lista principal, actualizando la base de datos instantáneamente.

5. Explicación del CRUD implementado

Se implementaron las cuatro operaciones fundamentales (Create, Read, Update, Delete) a través de la clase `ActividadViewModel`:

- Create (Crear): Se utiliza el método `.push()` de Firebase para generar un ID único y `.setValue()` para guardar el objeto `Actividad`.

- Read (Leer): Se implementó un `ValueEventListener` que escucha cambios en la referencia "actividades". Esto permite que la lista se actualice automáticamente para todos los usuarios sin refrescar la pantalla.
- Update (Actualizar): Se reutiliza el método `.setValue()` apuntando al ID específico de la actividad, reemplazando los valores antiguos por los nuevos capturados en el formulario.
- Delete (Eliminar): Se utiliza el método `.removeValue()` sobre la referencia del ID seleccionado para borrar el registro permanentemente.

6. Uso de Jetpack Compose

La interfaz se desarrolló íntegramente con Jetpack Compose, el toolkit moderno de Android. Se destaca el uso de:

- Scaffold: Para organizar la estructura básica con una `TopAppBar`.
- Gestión de Estado (`mutableStateOf` / `remember`): Crucial para que el formulario sea reactivo y guarde temporalmente lo que el usuario escribe.
- Componentes Material3: Uso de `OutlinedTextField` para una estética limpia, `Checkbox` para el estado de la tarea y `Button` para la acción de guardar.
- `LaunchedEffect`: Utilizado para detectar si se está en modo edición y sincronizar los datos del `ViewModel` con los campos de texto de la UI.

7. Uso de Firebase

Se seleccionó Firebase Realtime Database como backend debido a su capacidad de sincronización en milisegundos. A diferencia de las bases de datos tradicionales, Firebase nos permitió:

- Almacenar datos en formato JSON, ideal para la estructura de nuestra clase `Actividad`.
- Garantizar que, si el usuario cambia de dispositivo, sus tareas sigan disponibles al instante.
- Simplificar la conexión entre la nube y la app sin necesidad de configurar servidores complejos.

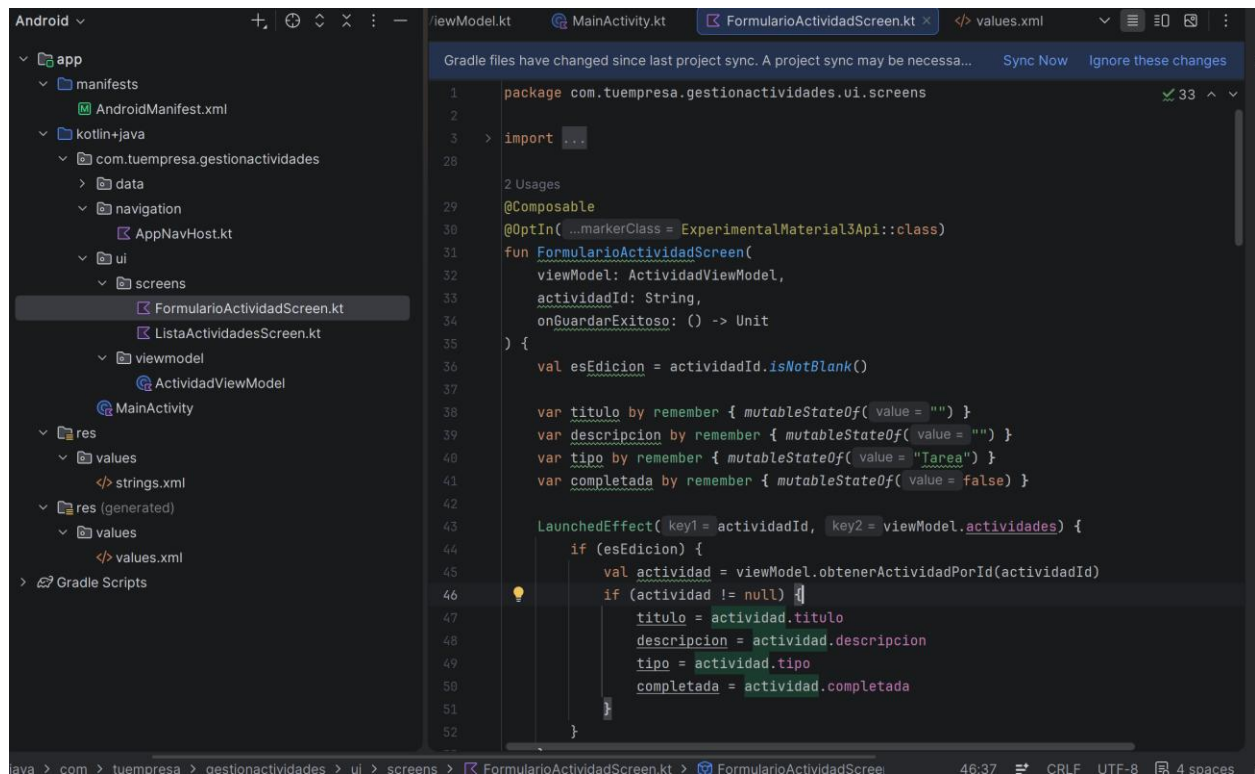
8. Uso de IA (Inteligencia Artificial)

La IA (Android Studio Bot) fue una pieza clave en el desarrollo:

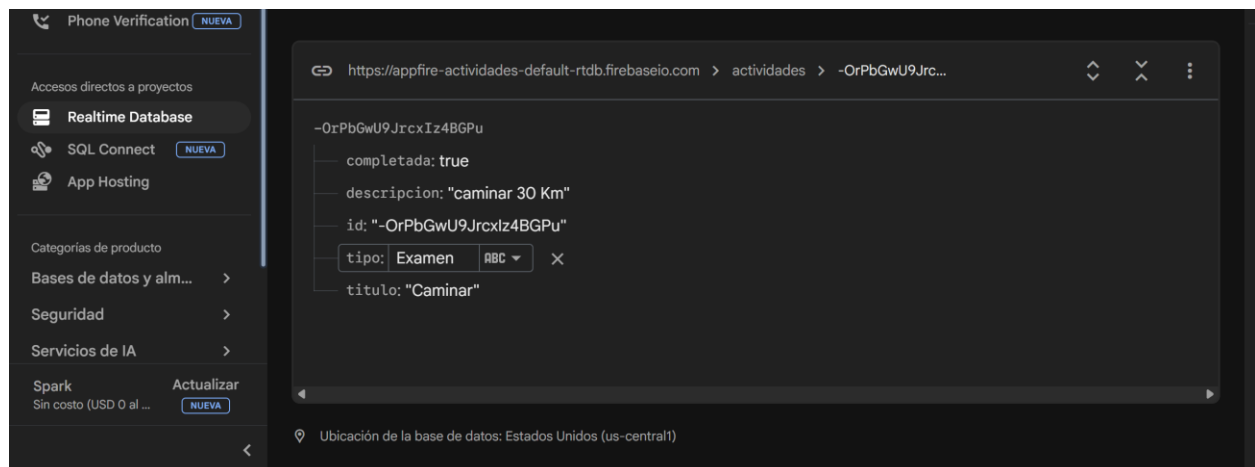
- Para qué la usamos: Para depurar errores de lógica en la navegación y entender cómo pasar el ID de una actividad entre pantallas. También nos ayudó a optimizar el código del `ViewModel`.
- En qué parte: Principalmente en la lógica del formulario y en la configuración de las dependencias de Google Services en el archivo `build.gradle.kts`.
- Aprendizaje: Comprendimos que la IA es un excelente tutor para explicar conceptos complejos como la "Recomposición" en Compose y el manejo de hilos asíncronos al consultar datos de internet.

9. Evidencias

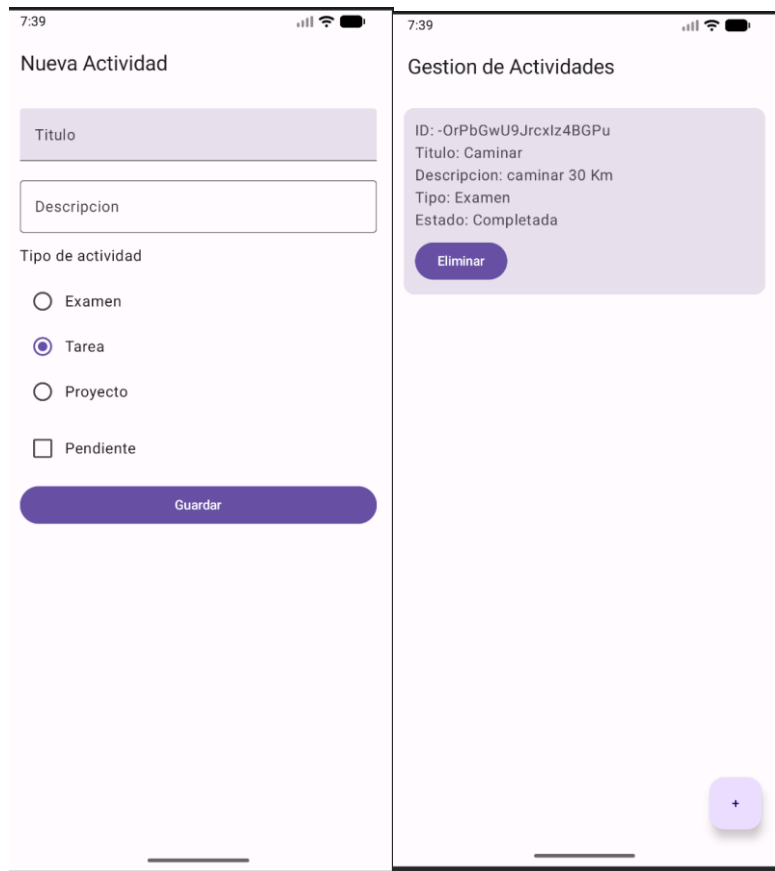
Captura 1: El código de `FormularioActividadScreen.kt`.



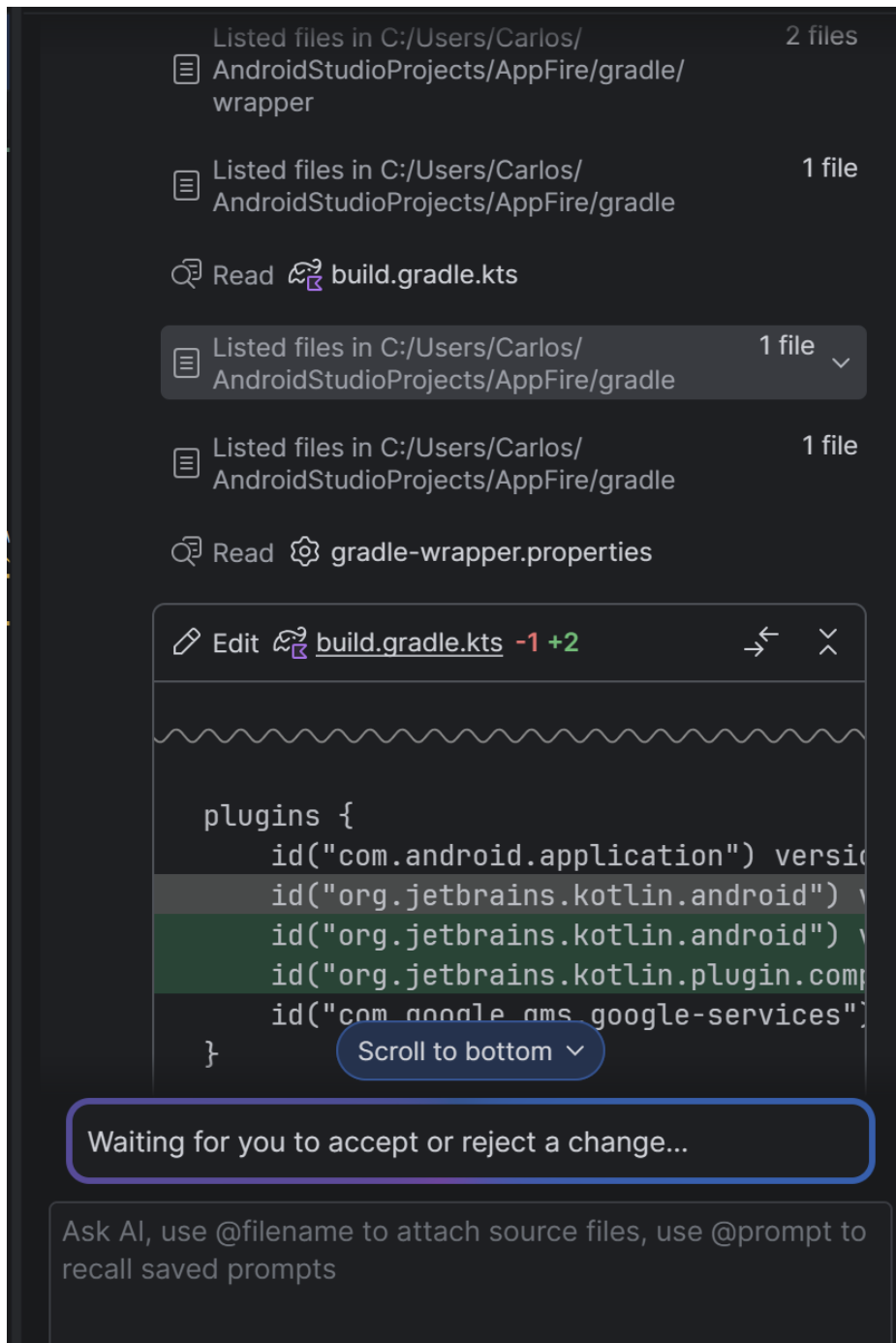
Captura 2: La consola de Firebase Realtime Database con los datos guardados.



Captura 3: La aplicación corriendo en el emulador (Lista y Formulario).



Captura 4: Una captura de este chat de IA como prueba de consulta.



10. Reflexión final

El desarrollo de AppFire ha sido un reto técnico que nos ha permitido integrar los pilares del desarrollo móvil actual. La combinación de Jetpack Compose y Firebase demuestra ser extremadamente potente para crear apps rápidas y escalables. Además, el uso ético y productivo de la IA nos permitió acelerar el aprendizaje y centrarnos en la experiencia del usuario. Concluimos que el manejo correcto de los "Estados" es el corazón de una app moderna y que la persistencia en la nube es ya un estándar indispensable.